Nobody Builds the Wall: LLM Agent Societies Under Market Rules Leave a Public Good Unbuilt

Draft 1, 2026-11-08 23:00 UTC. Slots marked `[E12]`, `[E09]`, `[E10]` fill from runs in progress. Every number traces to a file under `results/`; citations marked † are not yet cite-checked.

Abstract

We put the same 100 LLM agents (Gemma 4 26B), with the same personal goal, on islands that differ only in their economic rules, and let nightly mobs kill anyone outside a wall. Walls are a public good: they protect everyone and pay no one. Over 5 seeds of 12 days, capitalist islands built 0 walls and kept 5.0 of 100 people alive (range 3-7); a 20% tax that pays for wall building kept 8.8 alive but still built almost none (3 walls in 5 games). Socialist islands, where an elected leader sets job quotas, built 142 walls per game, lost no one to mobs, and kept 36.6 alive (30-48), losing the rest to starvation. Agents rested at similar rates under every rule set (0.24 vs 0.28), so the gap is provision of the public good, not effort. The socialist rules differ in two ways, shared ownership and a leader who assigns jobs, and these runs do not separate the two. In a one-year extension with seasons, births, elections and migration between a capitalist and a socialist island, the capitalist island died out in week 11-12 in the first seed, before the first assembly at week 13, so its agents never voted or moved. We repeat the games in real Minecraft with one mineflayer bot per agent: with 50 bots per island, both capitalist islands built no wall and lost everyone, while the socialist island built 70 walls and kept 35 of 50 alive. The same model with the same goal lived or died by the rules it ran under, so evaluating each agent alone does not predict what the population does.

1. Introduction

- Alignment work mostly evaluates one model. Deployed agents increasingly act in groups that share resources, and the rules of the group set which individually reasonable choices add up to collapse.
- Public goods are the textbook case (Samuelson 1954†; Olson 1965†): a good everyone uses and no one is paid to make is under-provided by self-interested actors unless an institution provides it.
- We ask whether LLM agents reproduce this, and whether the economic rule set decides it. The setup follows a public Minecraft experiment with 200 human players on a capitalist and a communist island (jjkay03, 2025), with LLM agents in place of people.
- Contributions:
  1. A preregistered island economy with three rule sets, one model and one goal (E08, 15 games of 100).
  2. A one-year, two-island extension where agents vote their rules, re-elect leaders and migrate (E12).
  3. A transfer check in real Minecraft with up to 50 bots per island (E09, E12-MC).
  4. Released code, preregistrations, ledgers and raw logs.

2. Related work

- **LLM agents in commons and public-goods games.** GovSim (Piatti et al. 2024†, arXiv 2404.16698) finds most LLM groups over-harvest a shared resource. Guzman Piedrahita et al. (2025†, arXiv 2506.23276) report that reasoning models free-ride in public-goods games. Both use small groups and fixed rules; we vary the rules and scale to 100 agents.
- **Many-agent Minecraft societies.** Project Sid (Altera.AL 2024†, arXiv 2411.00114) runs up to 1,000 agents and studies a tax vote; Civ50 (YC/Olam, 2026-10-29†) runs 50 agents once with no institutional variable. We hold the model and goal fixed and change only the institution.
- **Economic design with learning agents.** The AI Economist (Zheng et al. 2020†, arXiv 2004.13332) learns tax policy with RL agents; we use off-the-shelf LLMs and fixed, human-readable rules.
- **Planning and information.** Hayek (1945†) argues a central planner lacks agents' private knowledge. E10 tests this directly with private skills agents can misreport `[E10]`.

3. Setup

World (E08, `src/swarm/islands.py`). Each island has 100 people, the same start food and the same night rule. Each day every agent chooses a job: farm (food), chop (wood), mine (stone, sometimes ore, small cave-in risk),

build_wall (wood and stone into wall), or rest. Each night mobs attack; the wall covers the share min(1, wall / mob_need) of the island, and an agent outside cover may die. Mob need grows each night after two calm nights; walls decay. Agents who end a night without food starve. Every agent has the same goal text: comfort, defined as rest days plus own goods.

Rule sets. | island | ownership | market | who builds walls | |—|—|—|—| | capitalist | private goods | bank buys ore; food-for-coins auction | anyone who donates | | capitalist_tax | private goods | the same plus 20% tax on bank sales | paid 2 coins per wall from the tax | | socialist | common store, equal rations | none | elected leader sets daily quotas |

Model and protocol. Gemma 4 26B (A4B instruct) through the Adaption API, temperature default, one call per agent per decision. Preregistered before any main game (`experiments/E08_islands/prereg.md`); pilot games are excluded. 5 seeds per arm; each seed gives all islands the same people and traits.

Year extension (E12, `src/swarm/year.py`, prereg `experiments/E12_year/prereg.md`). Islands A (capitalist) and B (socialist), 24 people each, 52 weekly turns. Winter (weeks 40-52) rots a quarter of stored food each week and makes mobs 1.5x stronger. An island with ≥ 6 food per head adds 10% newborns. Quarterly assemblies (weeks 13, 26, 39) re-elect the leader on common-store islands. In the open arm, every citizen also sees both islands, votes the rules for the next quarter, and may move. The fixed arm keeps the rules and blocks migration. Predictions Y1-Y6 are in the prereg.

Minecraft (E09, `experiments/E09_minecraft/`). A Paper 1.21.4 server with one mineflayer bot per agent. The LLM layer is unchanged; work is done physically (bots harvest wheat, chop logs, mine stone for 45 s), the wall is real cobblestone on a ring, and nights summon real zombies the bots fight. Block counts convert to text units by a preregistered calibration. In E12-MC, births log in new bots and a migrant's bot logs out of one island and into the other.

4. Results

4.1 Rules decide who survives (E08)

Data: `results/E08/main_gemma.jsonl`, 15 games, 10.5k calls, $0.64. Means over seeds 0-4. | island | alive /100 | starved | killed by mobs | walls | chosen rest | wealth Gini | compliance | |—|—|—|—|—|—|—|—| | capitalist | 5.0 (3-7) | 36.4 | 53.2 | 0 | 0.239 | 0.237 | - | | capitalist_tax | 8.8 (4-14) | 23.8 | 61.0 | 0.6 | 0.248 | 0.216 | - | | socialist | 36.6 (30-48) | 58.6 | 0 | 142 | 0.282 | - | 0.823 |

prereg	prediction	result
H1 public good	socialist walls per capita ≥ 1.5x capitalist	holds: 142 vs 0 in every seed
H2 free riding	socialist chosen rest ≥ capitalist + 0.05	fails: +0.043 (higher in 4 of 5 seeds)
H3 inequality	capitalist Gini ≥ 0.3	fails: 0.237
H4 survival	survivors differ by ≥ 10	holds: +31.6, socialist higher in every seed
H5 compliance	≥ 0.6	holds: 0.823
H6 institution	tax walls ≥ 1.5x capitalist	nominal only: 3 walls in 5 games vs 0

- The capitalist islands die of mobs: no single agent gains from a wall, so none is built, while agents mine ore the bank buys. Paying for walls through a tax did not change this.
- The socialist islands die of hunger instead: the leader's quotas put people on walls and the common store runs short. The institution moved the cause of death, and the total, but did not remove it.
- Rest rates are close, so the socialist advantage is not more effort; it is effort aimed at the public good. H2 (free riding under equal rations) is not supported at the preregistered margin.

4.2 One year with votes and migration (E12)

Data: `results/E12/main.jsonl`, seed 0 (seeds 1-2 pending). 24 people per island at the start.

arm	alive A	alive B	rules at end (A, B)	switches	moves A to B, B to A	births	leader re-elected
fixed	0	20	capitalist, socialist	0	0, 0	0	3 of 3
open	0	6	capitalist, socialist	0	0, 0	0	3 of 3

- Island A died out before the first assembly in both arms (week 12 fixed, week 11 open), so no A citizen ever voted or chose to move. The first quarter is identical in the two arms, so the gap on B (6 vs 20 alive) arose before any vote: in the open arm B fell from 23 to 6 between weeks 6 and 12, to starvation, and stayed at 6 to week 52. It is run-to-run variation, not an effect of openness.
- On B, the 6 remaining people (open) kept the socialist rules and did not move to the empty island; the incumbent leader won all 3 elections in both arms (Y4 holds).
- Y1-Y3 and Y5 are untestable in seed 0 under the preregistered kill test's logic: the capitalist island had no one left to vote. Y6 is untestable: A is extinct by week 52.
- Design consequence: with the E08 day rules, a capitalist island of 24 lasts about 11 weeks, shorter than the first quarter. An assembly every 4 weeks would let A's citizens vote while they live. `[verdicts over seeds 0-2 when main_s1 and main_s2 arrive]`

Pilot note (not analysed): in a 14-week, 12-person pilot, island A died out in week 6 and two people from B moved onto it at the first assembly.

4.3 Label vs rules and hidden information (E10)

`[E10 neutral: survivors with the ideology words removed vs E08 labelled arms, same seeds]` `[E10 planned: under-report rate, plan efficiency; H7, H8]`

4.4 Transfer to Minecraft (E09, E12-MC)

- Cover sweep (E09, one night per cover level, 50 bots): deaths fall faster than linearly with wall cover (12/50 at cover 0, 3/33 at 0.5, 0/28 at 1.0); M3 (deaths at 0.5 ≥ 0.7x deaths at 0) not supported. Partial walls protect more than proportionally.
- Games at n = 50 per island, seed 0, one game per rule set (`results/E09/e09_mc.jsonl`):

island	survivors /50	starved	killed by zombies	walls
capitalist	0	4	46	0
capitalist_tax	0	5	45	0
socialist	35	15	0	70

- E12-MC (open arm, seed 0, one year): island A built no wall and died out by week 8, with 26 bots killed by zombies. Island B reached full wall cover, lost no one at night over 52 nights (1,788 zombies spawned), had 12 births and ended with 29 alive; one person moved from B to A. No rule switches. The text run of the same seed ended with 0 and 6 alive.
- In Minecraft the LLM layer is unchanged and the wall is placed by a server command once agents vote to build it; harvesting, mining and combat are physical.

5. Discussion

- **System-level alignment.** One model with one goal produced 5% or 37% survival depending only on rules written in a prompt. Evaluating that model alone would not predict either number.
- **Pricing the public good was not enough.** The tax arm paid for walls and still got almost none: the agents did not act on a small, diffuse wage when ore paid more directly.
- **Role-play threat.** Agents may follow "capitalism" or "socialism" as a script. E10 neutral removes the words while keeping the rules; `[E10 result]` decides how much of the gap is the label.

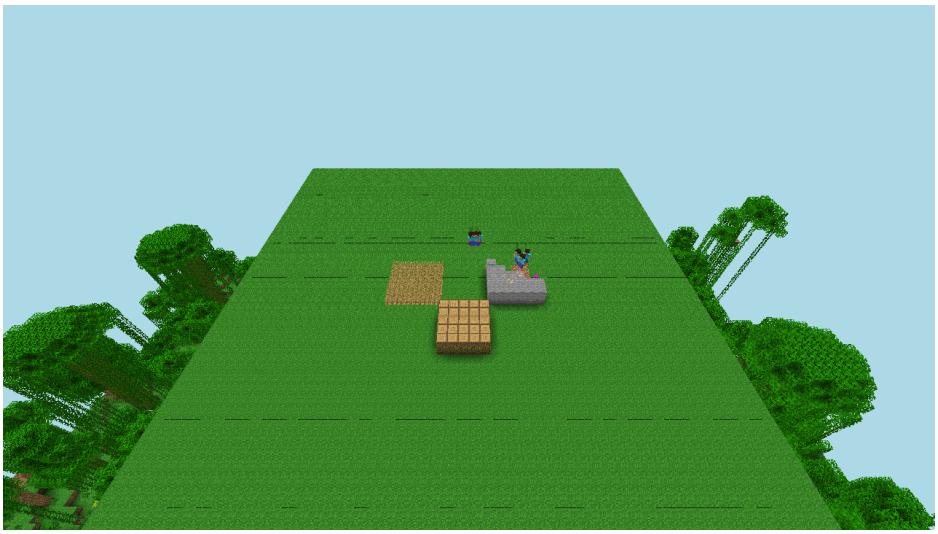


Figure 1: Island A (capitalist), E09 Minecraft, day: bots mining stone next to the wheat field and log block.

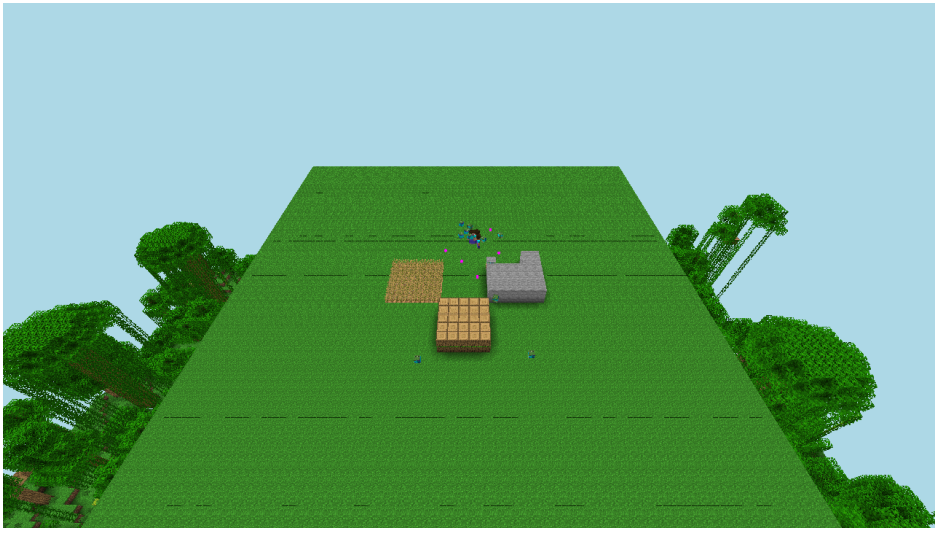


Figure 2: The same island at night: zombies on the bots at the island centre; no wall was built.

6. Limitations

- One model family (Gemma 4 26B) and one provider; other models may differ.
- 5 seeds per E08 arm and 3 per E12 arm; E12-MC and each E09 Minecraft game are n = 1.
- Rules are prompt-level; there is no enforcement beyond the simulator's accounting.
- The text world abstracts movement and combat. In Minecraft a day's harvest is capped by how fast the field regrows, and the bot standing closest takes most of it, which spreads private yields unevenly.
- `zombies_per_need` was calibrated at 50 bots; E12-MC runs 24 per island.
- E10 planned ran 5 seeds without the preregistered pilot. `[confirm deviations at submission]`

7. Dual use

The work shows how institutional rules change outcomes for groups of LLM agents. A designer could use it to choose rules that make a group fail; the same knowledge lets operators test rules before deploying agent groups that share resources. No human data; the Minecraft server is private.

Reproducibility

Code, preregistrations, raw logs, and the hash-chained run ledger (`papers/C/.compete/`) are in the repository. Runs: `experiments/E08_islands/run.py`, `experiments/E12_year/run.py`, `experiments/E09_minecraft/run.py`.